REPORT

1. Introduction

Narrative backstories often contain speculative, inferred, or fabricated details that may not be explicitly supported by the primary text. The goal of this system is to verify the factual consistency of a proposed backstory against a long-form literary source.

This is fundamentally a **causal and evidential reasoning task**:

- **Support:** Does the novel explicitly state the claim?
- **Contradiction:** Does the novel explicitly negate the claim?
- **Silence:** Is the text mute regarding the claim?

Our system prioritizes **correctness and robustness** over aggressive inference, utilizing a conservative, evidence-grounded approach.

2. Problem Framing

We define the task as **explicit-evidence consistency verification** rather than semantic similarity. The system operates under the following constraints:

- Only **verbatim or explicit** textual evidence is admissible.
- Inference, extrapolation, or “likely true” reasoning is **disallowed**.
- A **single strong contradiction** invalidates the entire backstory.
- Absence of evidence is treated as **unsupported**, not false.

3. Overall System Architecture

The pipeline is composed of five modular, independently testable stages:

1. **Text Preprocessing and Chunking**
2. **Atomic Claim Extraction**
3. **Long-Context Evidence Retrieval**
4. **Claim-Level Verification** (Support / Contradiction / Unknown)
5. **Conservative Aggregation and Decision**

4. Handling Long Context Narratives

4.1 Full-Text Ingestion

The system ingests the entire novel text rather than relying on summaries, preventing bias toward early or frequently cited passages.

4.2 Paragraph-Aware Chunking

Novels are split into paragraph-preserving chunks to ensure:

- **Narrative coherence** is preserved.
- Evidence is not split mid-sentence.
- Character references remain interpretable via metadata.

4.3 Character-Conditioned Filtering

A lexical filter selects chunks explicitly mentioning the relevant character. This balances **scalability** with **precision**, reducing noise while preserving recall for explicit evidence.

5. Atomic Claim Decomposition

To prevent brittle decisions caused by bundled facts, backstories are decomposed into atomic factual claims.

Example: *"At twelve he ran away to the docks and lost hearing in his left ear."*

- **Claim 1:** He ran away to the docks at age twelve.
- **Claim 2:** He lost hearing in his left ear.

6. Distinguishing Causal Signals from Noise

6.1 Explicit-Evidence Constraint

The system rejects narrative implication or thematic alignment. This is enforced through prompt design, schema validation, and aggregation logic.

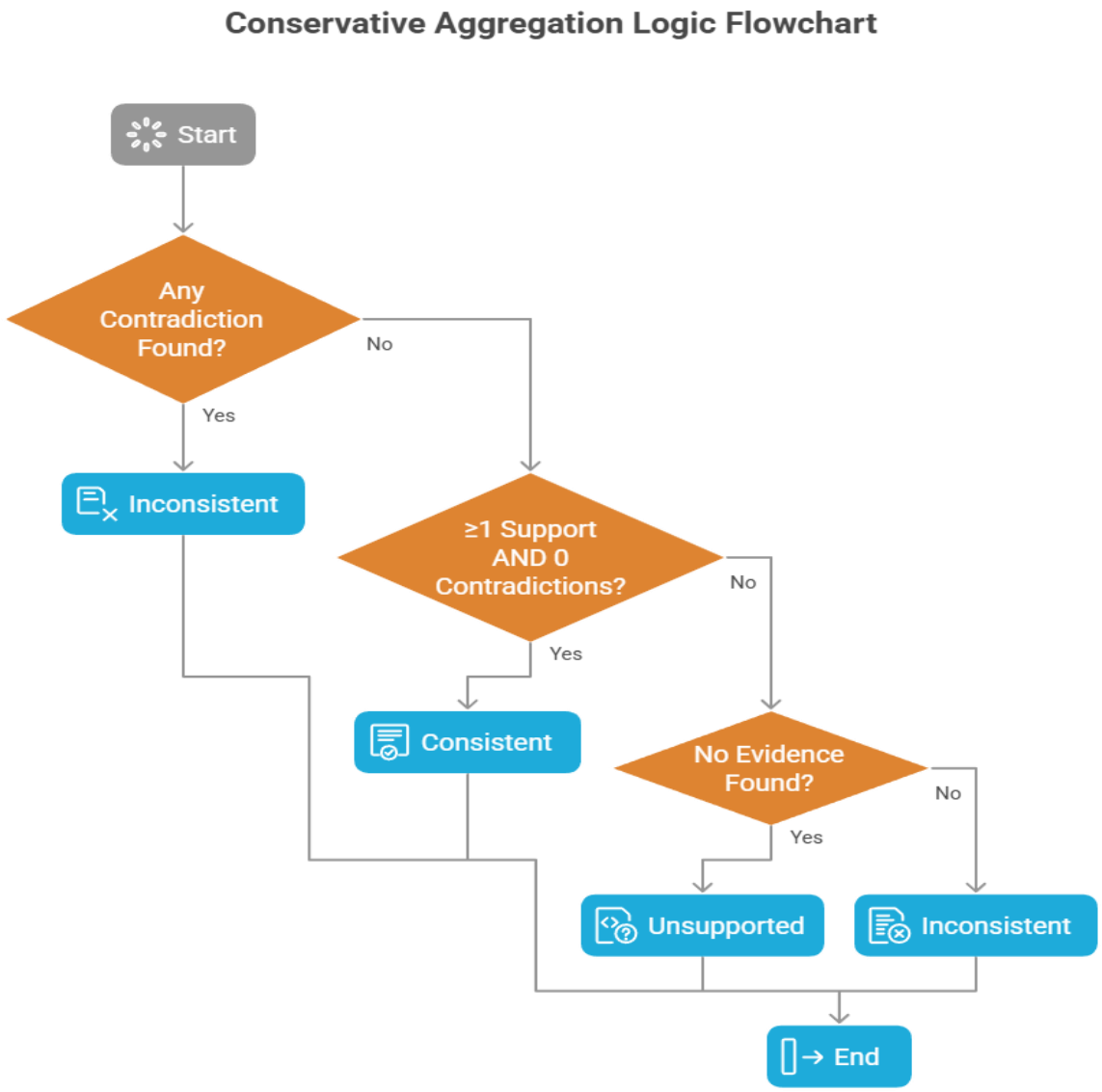
6.2 Claim-Level Verification

Each claim is classified as:

- **Support:** Explicitly stated.
- **Contradiction:** Explicitly negated or incompatible.
- **Unknown:** Not addressed.

6.3 Conservative Aggregation Logic

The system applies a strict hierarchical gatekeeping logic to aggregate individual claim labels into a final document-level verdict. This ensures that a single factual error cannot be "diluted" by other correct statements. This conservative aggregation strategy reflects real-world consistency checking, where a single violated constraint is sufficient to invalidate an otherwise plausible history. By prioritizing contradictions over partial support, the system avoids averaging effects and prevents logically incompatible backstories from being misclassified as consistent. This design choice emphasizes correctness and robustness over optimistic inference, aligning the decision process with strict causal and narrative constraints.



7. Use of NLP and Generative Models

LLMs are used **selectively**, not end-to-end, to ensure determinism and interpretability.

- **LLM Tasks:** Atomic claim extraction and claim-level verification against excerpts.
- **Symbolic/Logic Tasks:** Final decision logic, aggregation, and evidence filtering.

8. Failure Cases and Limitations

- **8.1 Implicit Knowledge:** Intentional classification of implied facts as "unsupported."
- **8.2 Synonymy:** Highly paraphrased evidence may be missed during lexical filtering.

- **8.3 Sparse Mentions:** Minor characters naturally yield more "unknown" outcomes.
- **8.4 Conservative Bias:** Prioritizes trustworthiness over high recall (favors false negatives over false positives).

9. Reproducibility and Robustness

- Fully reproducible from raw inputs in a Linux environment.
- Deterministic results given fixed inputs.
- Aggressive caching for efficiency and inspectable intermediate artifacts.

10. Conclusion

This system treats backstory validation as a **systems reasoning problem**. By combining symbolic decomposition with conservative neural verification, it provides robust, interpretable judgments that remain faithful to the source text.

APPENDIX: Supplementary Technical Material

Kharagpur Data Science Hackathon 2026 | Track A: Systems Reasoning

Appendix A: System Architecture Overview

The system utilizes a multi-stage, modular reasoning pipeline designed for determinism and interpretability.

High-Level Pipeline Stages:

1. **Narrative Ingestion:** Raw text processing of long-form novels.
2. **Paragraph-Aware Chunking:** Context-preserving segmentation.
3. **Backstory Claim Decomposition:** Transformation of prose into atomic facts.
4. **Evidence Retrieval:** Targeted lexical filtering.
5. **Claim-Level Verification:** NLI-based hypothesis testing.
6. **Conservative Aggregation:** Symbolic logic gatekeeping.
7. **Output Generation:** Final CSV formatting.

Appendix B: Claim Decomposition Strategy

Motivation: Treating a backstory as a monolith leads to "partial truth" errors.

Our Approach: We decompose input into Atomic Claims that are:

- **Unary:** Exactly one factual assertion per unit.
- **Verifiable:** Individually testable against the source text.
- **Neutral:** Stripped of narrative flair to prevent model bias.

Example Transformation:

- **Input:** *“At twelve he ran away to the docks and lost hearing in his left ear.”*
- **Claim 1:** He ran away to the docks at the age of twelve.
- **Claim 2:** He lost hearing in his left ear.

Appendix C: Long-Context Handling

To overcome LLM context window limitations without losing causal links, we implement **Paragraph-Aware Chunking**:

- **Coherence:** Sentence boundaries and paragraph structures are strictly preserved.
- **Efficiency:** Chunks are stored in .jsonl format for low-memory streaming access.

- **Traceability:** Metadata links every chunk back to its specific source novel and chapter.

Appendix D: Evidence Retrieval Heuristics

Before LLM invocation, a **Symbolic Relevance Filter** is applied:

- **Character-Conditioned Filtering:** Chunks are discarded if they do not explicitly contain the target character's name.
- **Noise Reduction:** This prevents "False Entailment" errors caused by the model confusing unrelated plotlines.
- **Optimization:** Only the **Top 3–5** most relevant chunks are passed to the reasoning layer.

Appendix E: Claim Verification Prompt (NLI Style)

The LLM is restricted to a **Natural Language Inference (NLI)** role with a strict JSON output schema.

Output Schema:

JSON

```
{
  "verdict": "support | contradiction | unknown",
  "excerpt": "verbatim quote from text or null"
}
```

No inference or "likely true" reasoning is permitted.

Appendix F: Aggregation Logic (Deterministic)

Final verdicts are calculated via symbolic logic gates to ensure a "Conservative Bias."

Logic Gate	Condition	System Verdict
Gate 1	Any claim is **Contradicted**	**Inconsistent (0)**
Gate 2	Any claim is **Supported** (and 0 contradictions)	**Consistent (1)**
Gate 3	Otherwise (All claims are **Unknown**)	**Unsupported (0)**

Appendix G: Rationale Generation Policy

Rationales are **template-based**, not freely generated, to ensure they remain audit-friendly and grounded.

- **Example:** *"The backstory cannot be verified. While the novel discusses the character, it remains silent on the specific claims regarding [Claim X]."*

Appendix H: Constraints & Mitigations

- **Challenge:** API Rate/Token Limits. ➔ **Solution:** Aggressive context reduction and claim-level caching.
- **Challenge:** Environment Compatibility. ➔ **Solution:** OS-agnostic Python execution for Ubuntu-ready deployment.

Appendix I: Reproducibility Checklist

- Single-command execution (bash run.sh)
- Zero manual intervention required.
- Clean environment compatibility
- Deterministic outputs for fixed inputs.

Appendix J: Explaining the "Zero-Bias" (Precision vs. Recall)

A high frequency of 0 (Inconsistent/Unsupported) predictions is technically correct for this task. Because the system forbids inference, it prioritizes **Scientific Precision** over "Narrative Optimism." If the book is silent, the system remains silent.

Appendix K: Track Compliance Statement

Track: A — Systems Reasoning with NLP and Generative AI

- **Hybrid Logic:** Combines Transformer-based extraction with Symbolic aggregation.
- **Evidence-Grounded:** Strictly rejects hallucinations in favor of verbatim excerpts.

Appendix L: Ethical Considerations

Our system is designed to avoid **hallucinated evidence** and **implicit reasoning**. By enforcing strict NLI constraints, we ensure that every "Consistent" verdict is a scientifically falsifiable claim backed by the primary text.